

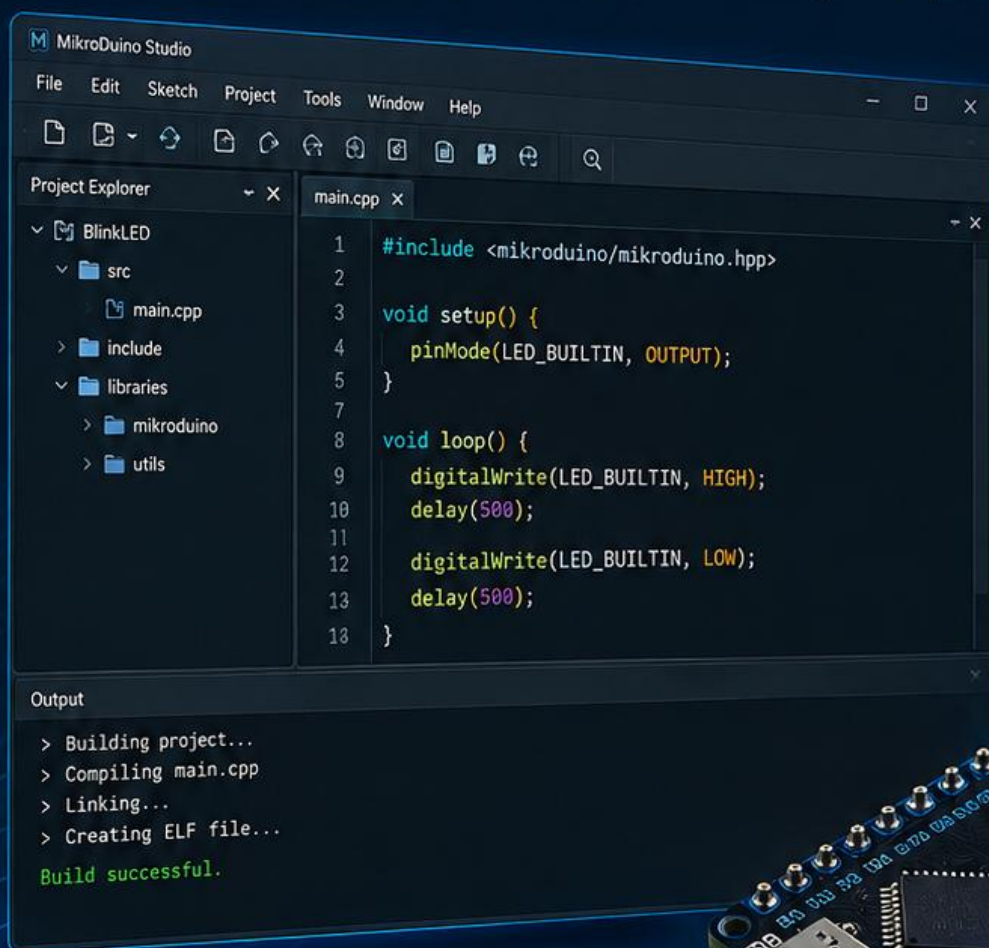


MikroDuino STUDIO

THE PROFESSIONAL IDE FOR AVR MICROCONTROLLERS

USER MANUAL

Write • Build • Debug • Upload



Intelligent Code
Editor



Powerful Debug
Tools



AVR GCC
Compiler



Easy Upload
& Serial Monitor



C/C++
Development



Optimized
Libraries



Arduino
Compatible



Fast. Reliable.
Productive.



DESIGNED FOR MAKERS. BUILT FOR PROFESSIONALS.

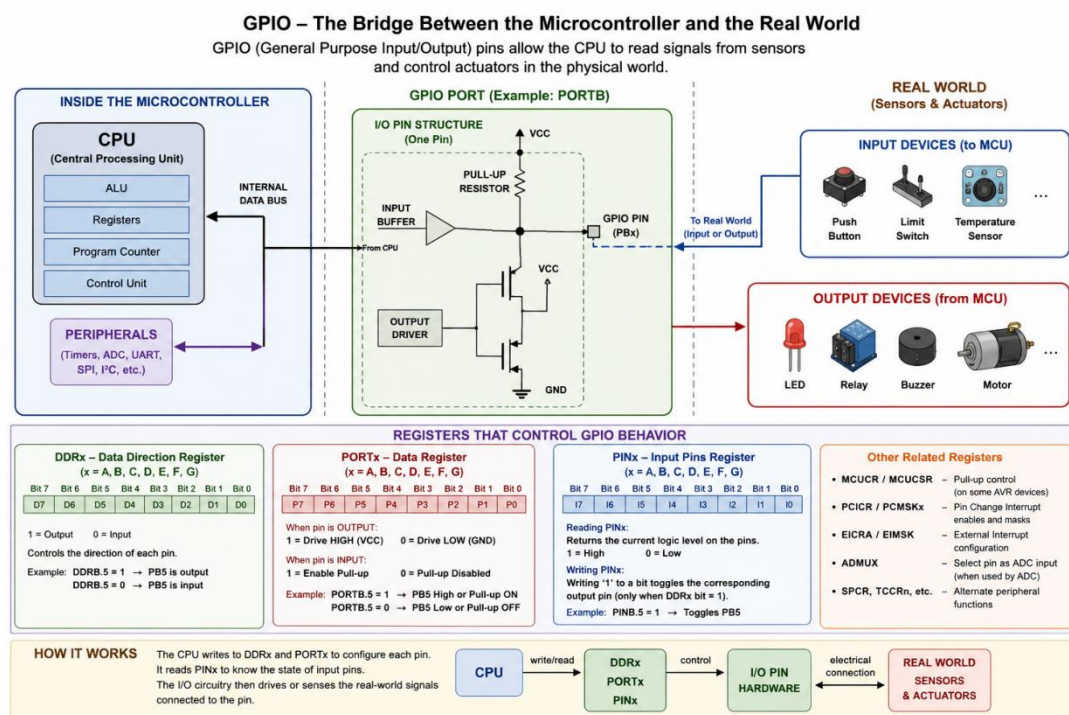
mikroduino.com

General Purpose Input Output System | GPIO

The Digital Hands and Eyes of a Microcontroller

Imagine asking a person to solve a complex mathematical problem while blindfolded, unable to hear, and unable to move their hands. Even if that person possessed extraordinary intelligence, they would still be unable to interact with the world around them. They could think, but they could neither observe nor perform any useful task.

A microcontroller faces exactly the same challenge. Inside every AVR microcontroller lies an incredibly capable Central Processing Unit (CPU). It can execute millions of instructions every second, perform arithmetic calculations, make decisions, store information in memory, and run sophisticated algorithms. Yet without some way to communicate with the outside world, all of this processing power would remain trapped inside a tiny silicon chip.



A microcontroller therefore needs **eyes** to observe what is happening around it and **hands** to influence its surroundings. This is precisely the purpose of the **General Purpose Input/Output**, more commonly known as **GPIO**.

GPIO forms the bridge between the digital world inside the microcontroller and the physical world around it. Whenever a push button is pressed, an LED lights up, a relay switches on, a sensor reports a measurement, or a motor begins to rotate, GPIO

is involved somewhere in the process. Almost every embedded application, no matter how simple or sophisticated, begins with GPIO.

Whether you are building a toy robot, a medical monitoring device, a home automation controller, an industrial machine, or even a satellite, one of the very first tasks performed by the firmware is configuring GPIO pins. It is impossible to write useful embedded software without understanding GPIO.

This chapter therefore lays the foundation upon which almost every subsequent chapter of this book will build.

Why Does Every Microcontroller Have GPIO?

Different microcontrollers vary tremendously in their processing power. Some 8-bit AVR devices execute a few million instructions per second. Modern 32-bit ARM processors execute billions of instructions every second. Some contain only a few kilobytes of memory, while others contain several gigabytes. Some include graphics processors, wireless communication, encryption hardware, artificial intelligence accelerators, or floating-point units.

Despite these differences, one feature is common to virtually every microcontroller ever manufactured:

They all provide General Purpose Input/Output pins.

The reason is surprisingly simple. No matter how intelligent a processor becomes, it remains useless unless it can communicate with the physical world. Every embedded system must perform two fundamental tasks:

- Receive information.
- Produce an action.

GPIO provides the simplest possible mechanism for both. An input pin allows the microcontroller to determine whether a voltage is HIGH or LOW. An output pin allows the microcontroller to generate either a HIGH or LOW voltage. These two simple operations are sufficient to build astonishingly complex systems.

Consider a digital thermometer. The temperature sensor sends information to the microcontroller through one or more GPIO pins. After processing the measurement, the microcontroller updates an LCD display using another group of GPIO pins. If the temperature exceeds a limit, another GPIO pin activates a buzzer. Everything revolves around GPIO.

GPIO Is Everywhere

Many beginners imagine GPIO to be useful only for turning LEDs on and off. In reality, LEDs are merely the traditional "Hello World" example of embedded programming. GPIO performs far more important roles in practical applications. A few examples include:

Reading Human Inputs

GPIO detects when a user presses a push button, rotates an encoder, flips a switch, or inserts a connector.

Examples include:

- ✓ Calculator keyboards
- ✓ Television remote receivers
- ✓ Elevator buttons
- ✓ Washing machine controls
- ✓ Medical equipment keypads
- ✓ Industrial control panels

Controlling Indicators

GPIO drives visual indicators that inform the user about the system's status.

Examples include:

- ✓ LEDs
- ✓ Seven-segment displays
- ✓ LCD control lines
- ✓ Warning lamps
- ✓ Status indicators
- ✓ Traffic signals

Driving External Devices

A GPIO pin can directly control low-current devices or indirectly control larger equipment through driver circuits.

Examples include:

- ✓ Relays
- ✓ Buzzers
- ✓ Fans
- ✓ Motors
- ✓ Solenoid valves
- ✓ Power MOSFETs
- ✓ Opto-isolators

Communicating with Sensors

Many sensors produce simple digital outputs. GPIO can directly read devices such as:

- ✓ Limit switches
- ✓ Infrared obstacle sensors
- ✓ Hall-effect sensors
- ✓ Motion detectors
- ✓ Float switches
- ✓ Rain sensors
- ✓ Flame detectors

Software Communication Protocols

Even sophisticated communication standards often begin with ordinary GPIO pins. Protocols such as:

- ✓ SPI
- ✓ I²C
- ✓ UART
- ✓ OneWire
- ✓ Bit-banged serial communication

all rely on GPIO hardware. Specialized peripherals like I2C and SPI merely automate what GPIO is fundamentally capable of doing.

Timing and Measurement

GPIO can also participate in measuring time.

Applications include:

- ✓ Pulse counting
- ✓ Frequency measurement
- ✓ Pulse-width measurement
- ✓ Rotary encoder reading
- ✓ Event detection
- ✓ External interrupts

A GPIO Pin Is More Than Just a wire

At first glance, a GPIO pin appears to be nothing more than a tiny metal connection emerging from the microcontroller package. In reality, every GPIO pin contains an entire electronic circuit hidden inside the chip.

Behind every pin are:

- ✓ Input protection circuitry
- ✓ Input buffers
- ✓ Output drivers
- ✓ Direction control logic
- ✓ Pull-up resistors
- ✓ Switching transistors
- ✓ Control registers

These internal components allow a single physical pin to perform many different functions. For example, a single AVR pin might simultaneously be capable of acting as:

- ✓ A digital input
- ✓ A digital output
- ✓ An ADC input

- ✓ An external interrupt
- ✓ A timer output
- ✓ An SPI signal
- ✓ A UART signal
- ✓ An I²C line

The software determines which function is currently active.

Learning GPIO therefore means learning how to control this hidden hardware.

GPIO Fundamentals and AVR Hardware

Before we begin programming, we must first understand how the AVR itself views its input and output pins.

Many beginners immediately start memorizing statements such as

```
DDRB |= (1 << DDB5);
```

without understanding what actually happens inside the microcontroller. This approach eventually leads to confusion. Instead, we shall begin from the hardware itself and gradually work our way upward until the software becomes almost self-explanatory.

By the end of this part of the chapter, you will understand not only **how** to configure GPIO pins, but **why** every instruction works.

What Is a GPIO Pin?

A **GPIO pin** is a programmable electrical connection between the microcontroller and the outside world. Unlike dedicated pins that always perform one specific task, a GPIO pin is flexible. Software can configure it to perform different jobs depending upon the needs of the application. The term **General Purpose** simply means that the pin is not permanently assigned to one function. The term **Input/Output** means that it can either receive signals or generate signals.

Every GPIO pin therefore operates in one of two fundamental directions.

Input Mode

The pin observes the voltage applied by an external device. The microcontroller becomes a listener. It does not attempt to drive the wire. Instead, it simply watches whether the voltage is HIGH or LOW.

Typical examples include reading:

- ✓ Push buttons
- ✓ Switches
- ✓ Sensors
- ✓ Digital outputs from other ICs

Output Mode

The microcontroller actively controls the voltage present on the pin. The pin becomes an electrical source capable of driving external circuitry.

Examples include:

- ✓ Lighting LEDs
- ✓ Controlling relays
- ✓ Operating buzzers
- ✓ Sending serial data
- ✓ Producing timing pulses

One physical pin cannot safely behave as both an input and an output simultaneously. The firmware must decide which role is appropriate at any given time.

This simple choice—listening or driving—is the foundation of all GPIO programming. In the sections that follow, we will open the "black box" of an AVR GPIO pin, examine the circuitry hidden behind it, and learn how the **DDRx**, **PORTx**, and **PINx** registers control every aspect of its behavior.

Register Programming and Bit Manipulation

After understanding the physical hardware behind an AVR GPIO pin, we are now ready to learn how software actually controls that hardware.

One of the greatest strengths of the AVR architecture is its simplicity. Unlike many modern microcontrollers that hide hardware behind multiple abstraction layers, the AVR allows a programmer to communicate directly with the hardware through a small number of registers.

This direct control is one of the reasons AVR has remained the favorite architecture for learning embedded systems. Nothing mysterious happens behind the scenes. Every instruction you write changes an actual hardware register, and every register controls a real electronic circuit inside the chip.

In this part of the chapter, we shall leave behind the conceptual view of GPIO and begin writing programs that manipulate the AVR hardware directly. Even though the MikroDuino SDK provides a clean, modern C++ interface, every embedded programmer should understand what happens underneath. Once you master register programming, every higher-level library becomes easier to understand, debug, and appreciate.

Registers – The Control Panel of the AVR

Imagine sitting inside the control room of a large factory.

On one wall is a massive control panel containing hundreds of switches, buttons, and indicator lamps. Each switch performs one specific task. One may start a conveyor belt, another may stop a motor, while another may sound an alarm.

The CPU inside the AVR works in a remarkably similar way. Instead of physical switches, it has hundreds of **registers**. Each register is simply a small storage location inside the microcontroller. Unlike ordinary RAM, however, these registers are directly connected to hardware circuits.



Changing a bit inside one of these registers immediately changes the behaviour of the hardware.

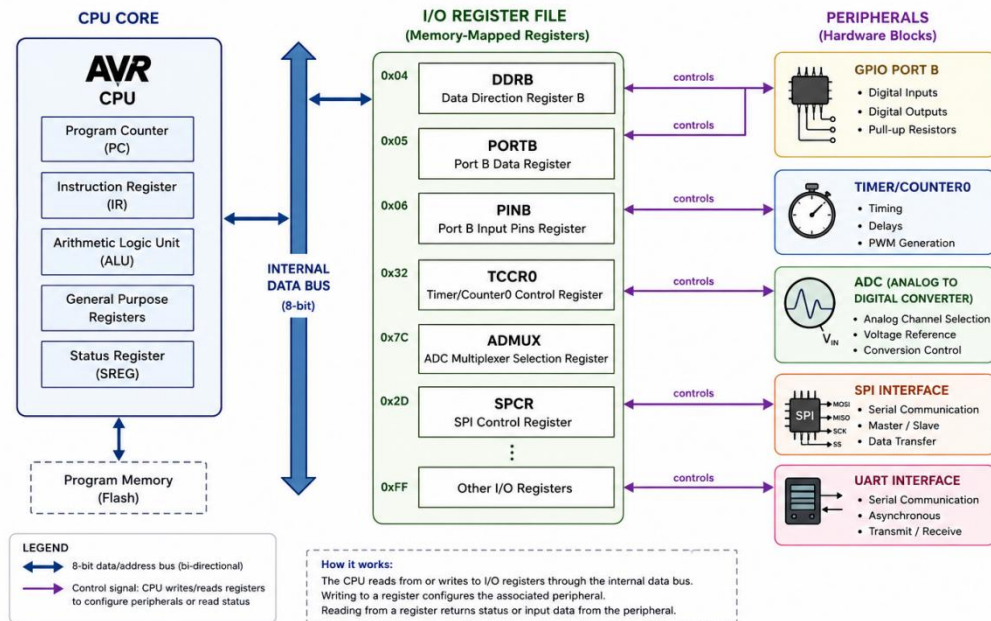
For example,

- ✓ one register starts a timer,
- ✓ another configures the ADC,
- ✓ another enables UART communication,
- ✓ another controls SPI,
- ✓ and three registers control every GPIO port.

A register is therefore much more than a memory location—it is a control interface to the hardware itself.

AVR Architecture – CPU, Registers and Peripherals

The CPU communicates with hardware peripherals through memory-mapped registers via the internal data bus.



What Exactly Is a Register?

A register is an 8-bit storage location. Since AVR is an 8-bit microcontroller, almost every hardware register contains exactly eight bits.

Bit7 Bit6 Bit5 Bit4 Bit3 Bit2 Bit1 Bit0

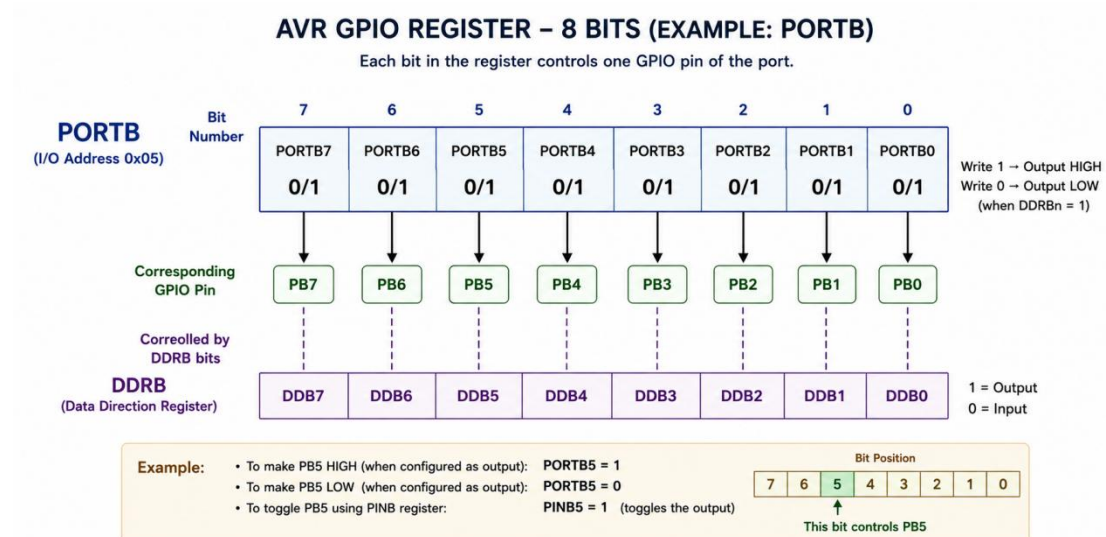
Each bit usually controls one feature. For GPIO registers, each bit corresponds to one physical pin.

For example,

DDRB
 Bit7 Bit6 Bit5 Bit4 Bit3 Bit2 Bit1 Bit0
 PB7 PB6 PB5 PB4 PB3 PB2 PB1 PB0

- ✓ Setting Bit 5 changes PB5.
- ✓ Setting Bit 2 changes PB2.

Changing one bit does not affect the others if performed correctly. This simple design makes AVR hardware extremely predictable.



The Three Registers Behind Every GPIO Port

Every AVR GPIO port is controlled by exactly three registers.

These are:

- ✓ DDR_x
- ✓ PORT_x
- ✓ PIN_x

where x represents the port letter.

For example, Port B uses:

DDRB
PORTB
PINB

Port C uses

DDRC
PORTC
PINC

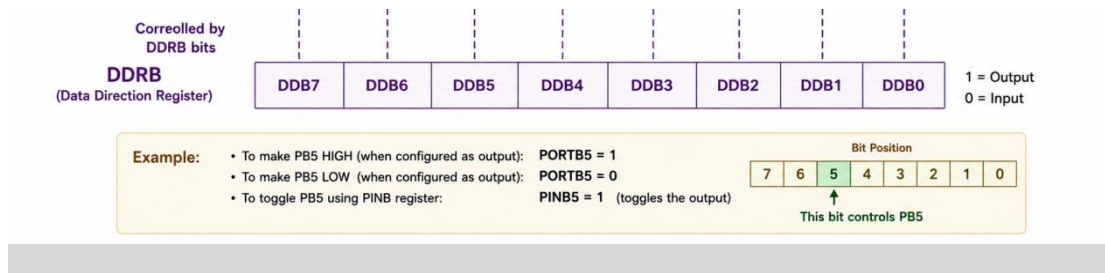
Port D uses

DDRD
PORTD
PIND

Regardless of which AVR you use, this organization remains almost identical. It is one of the reasons AVR programming is so portable.

DDR_x – Choosing the Direction

The **Data Direction Register** determines whether each pin behaves as an input or an output.



Each bit corresponds to one pin.

0 → Input
1 → Output

Suppose we want PB5 to become an output. We have to make bit 5 of DDRB register as 1.

In C, we write:

```
DDRB |= (1 << PB5);
```

What actually happens? Lets breakdown this statement.

First,

```
1 << PB5
```

produces

```
00100000
```

Then

```
OR
```

sets only Bit 5 while leaving every other bit unchanged. This is called a **Read-Modify-Write** operation.

One of the strengths of bitwise operations is that **multiple bits can be combined into a single mask** using the bitwise OR (`|`) operator. For example, if you want to configure **PB5, PB3, and PB1** as outputs simultaneously, you can write:

```
DDRB |= (1 << DDB5) | (1 << DDB3) | (1 << DDB1);
```

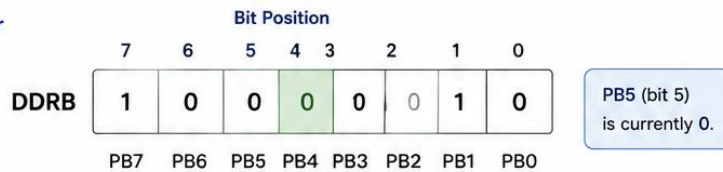
How the Statement `DDRB |= (1 << PB5);` Works

Step-by-step: Setting bit 5 of DDRB

1

Original DDRB Register

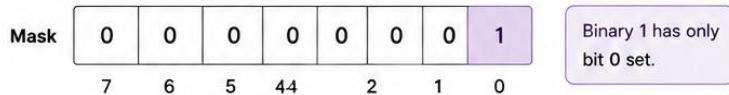
This is the current content of DDRB (before the operation).



2

Create Mask with 1

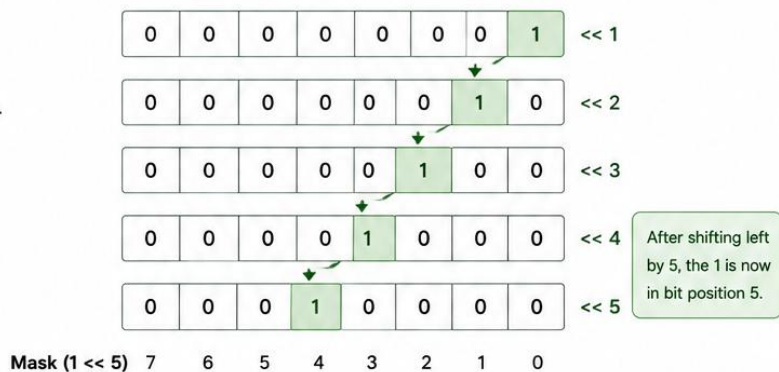
The number 1 in binary (8-bit representation).



3

Left Shift by 5

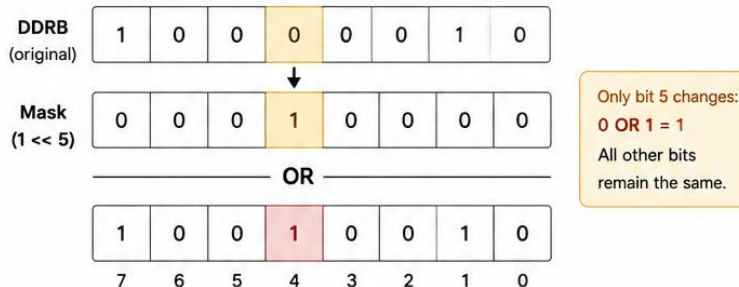
Perform `1 << 5`
(shift left by 5 positions).



4

Bitwise OR Operation

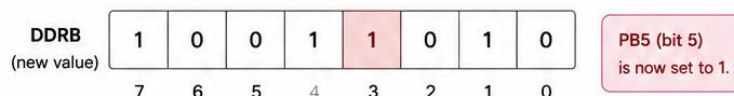
`DDRB |= (1 << 5)`
means
`DDRB = DDRB | (1 << 5)`



5

Result Stored Back in DDRB

The result of the OR operation is written back to DDRB.



6

Summary

`DDRB |= (1 << PB5);`
sets bit 5 of DDRB without affecting any other bits.



This creates three individual masks:

`(1 << DDB5) = 00100000`

`(1 << DDB3) = 00001000`

```
(1 << DDB1) = 00000010
----- OR
          00101010
```

The processor then performs:

```
DDRB = DDRB | 00101010;
```

Only bits **5**, **3**, and **1** are set to **1**, while all other bits remain unchanged.

Another example

Configure the lower four pins (PB0–PB3) as outputs:

```
DDRB |= (1 << DDB0) |
        (1 << DDB1) |
        (1 << DDB2) |
        (1 << DDB3);
```

Equivalent mask:

```
00001111.
```

One thing that is very interesting, the literal PB5 and others are defined in the AVR/IO.h library as a constant number like PB5=5, all these operations like (1<<PB5) is treated like (1<<5) by the compiler and the mask is generated at the time of compilation. So at run time this processing is not required. However in your code writing the statements like:

```
DDRB |= (1 << DDB5) | (1 << DDB3) | (1 << DDB1);
```

makes it extremely easy for programmer to later understand what is happening in the code.

Configure all pins as outputs

You can write

```
DDRB = 0xFF;
```

or

```
DDRB |= 0xFF;
```

or, for educational purposes,

```
DDRB |= (1 << DDB0) |
        (1 << DDB1) |
        (1 << DDB2) |
        (1 << DDB3) |
        (1 << DDB4) |
        (1 << DDB5) |
        (1 << DDB6) |
        (1 << DDB7);
```

Although the last version is verbose, it clearly shows which bits are being manipulated and is useful when first learning bit masks.

A Cleaner Approach

When the desired bit pattern is already known, you can combine the shifts into a single binary or hexadecimal constant:

```
// PB5, PB3 and PB1 as outputs
DDRB |= 0b00101010;
```

or

```
DDRB |= 0x2A;
```

However, **using named bit positions (DDB5, DDB3, etc.) is generally recommended** because the code is self-documenting. A reader can immediately see which pins are being configured without having to decode the binary or hexadecimal value. This is why the AVR datasheets and professional embedded codebases commonly use expressions like:

```
DDRB |= (1 << DDB5) | (1 << DDB3) | (1 << DDB1);
```

rather than hard-coded numeric constants.

PORTx – Controlling the Output

Once a pin has become an output, PORTx determines its voltage.

```
PORTB
Bit = 1
Output HIGH

Bit = 0
Output LOW
```

Example

```
PORTB |= (1 << PB5); // Set bit
```

LED ON.

```
PORTB &= ~(1 << PB5); // clear bit
```

LED OFF.

Clearing a Bit in a Register Using Bitwise AND (&)

Statement: `DDRB &= ~(1 << DDB5);` // Clear bit 5 of DDRB

1

Original Register (DDRB)
This is the current content of DDRB before the operation.

Bit Position

7	6	5	4	3	2	1	0
1	0	1	0	1	1	0	1
PB7	PB6	PB5	PB4	PB3	PB2	PB1	PB0

We want to clear bit 5 (PB5). It is currently 1.

2

Create Mask for the Bit
Shift 1 left by 5 positions.

1 << 5

7	6	5	4	3	2	1	0
0	0	1	0	0	0	0	0

Mask has 1 at bit 5.

3

Invert the Mask
Use bitwise NOT (~) to make all bits 1 except bit 5.

~(1 << 5)

7	6	5	4	3	2	1	0
1	1	0	1	1	1	1	1

Bit 5 is 0, all other bits are 1. This mask will clear bit 5 when used with AND.

4

Perform AND Operation
`DDRB &= ~(1 << 5)`

DDRB	1	0	1	0	1	1	0	1	(Original)
~(1 << 5)	1	1	0	1	1	1	1	1	(Mask)
	&								
Result	1	0	0	0	1	1	0	1	(After AND)

At bit 5:
1 AND 0 = 0
All other bits remain unchanged.

5

Result Stored Back in DDRB
The result is written back to DDRB.

DDRB

1	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---

Bit 5 (PB5) is now 0. It has been cleared.

Summary
To clear bit n in a register:

`REG &= ~(1 << n);`

Create a mask with 0 at bit n and 1s elsewhere, then AND it with the register.

1 = Bit is set
0 = Bit is cleared

Just like using OR to set bit we can make an inverted mask with 0 in the desired position and the AND it with native register, this will set the required bit to '0'.

Summary of Common Bit Manipulation Operations

Throughout this chapter, we have learned that every AVR register is an 8-bit value, with each bit controlling a specific hardware feature. Rather than writing an entire register every time, it is usually preferable to modify only the required bit while leaving all other bits unchanged.

The following operations form the foundation of almost all embedded programming.

Setting a Bit (Make a Bit = 1)

To set a bit means to change its value to 1 without affecting any other bits in the register.

This is accomplished using the **bitwise OR (|)** operator.

Syntax

```
REGISTER |= (1 << BIT);
```

Example

Configure PB5 as an output by setting bit 5 of the DDRB register.

```
DDRB |= (1 << DDB5);
```

What happens?

- ✓ Create a mask with a single 1 at the desired bit position.
- ✓ Perform a bitwise OR with the register.
- ✓ The selected bit becomes 1.
- ✓ All remaining bits retain their previous values.

Clearing a Bit (Make a Bit = 0)

To clear a bit means to force its value to **0** while preserving every other bit.

This is achieved using the **bitwise AND (&)** operator together with the **bitwise NOT (~)** operator.

Syntax

```
REGISTER &= ~(1 << BIT);
```

Example

Configure PB5 as an input by clearing bit 5 of DDRB.

```
DDRB &= ~(1 << DDB5);
```

What happens?

- ✓ Create a mask with a single 1.
- ✓ Invert the mask using ~, producing zeros only at the desired bit position.
- ✓ AND the register with this mask.
- ✓ The selected bit becomes 0.
- ✓ All remaining bits remain unchanged.

Toggling a Bit (Invert Its Value)

Sometimes we do not know whether a bit is currently 0 or 1. Instead of setting or clearing it explicitly, we simply want to reverse its current state.

A toggle operation changes

0 → 1

1 → 0

The software method uses the **bitwise XOR (^)** operator.

Syntax

```
REGISTER ^= (1 << BIT);
```

Example

Toggle PB5.

```
PORTB ^= (1 << PORTB5);
```

If PB5 was HIGH, it becomes LOW. If PB5 was LOW, it becomes HIGH.

AVR Hardware Toggle

AVR microcontrollers provide an even more efficient method for toggling an output pin.

Writing a **1** to the corresponding bit in the **PINx** register automatically toggles the output.

```
PINB = (1 << PINB5);
```

Unlike the software XOR method, this operation is performed entirely by the hardware. It executes in a single instruction, is atomic, and cannot be interrupted midway. For this reason, the MikroDuino SDK implements `GPIO::toggle()` using this AVR hardware feature rather than the XOR method.

Reading (Testing) a Bit

Frequently, a program needs to determine whether a bit is currently set or cleared.

This is done using the **bitwise AND (&)** operator.

Syntax

```
if (REGISTER & (1 << BIT))
{
    // Bit is 1
}
```

Example

Check whether PD2 is HIGH.

```
if (PIND & (1 << PIND2))
{
    // PD2 is HIGH
}
```

If the result of the AND operation is non-zero, the selected bit is set.

Setting Multiple Bits Simultaneously

Several bits can be manipulated in a single statement by combining masks with the bitwise OR operator.

```
DDRB |= (1 << DDB5) |
        (1 << DDB3) |
        (1 << DDB1);
```

This configures PB5, PB3, and PB1 as outputs while leaving all other pins unchanged.

Quick Reference Table

Operation	Purpose	Syntax	Example
Set a bit	Make the selected bit 1	<code>REGISTER</code>	<code>= (1 << BIT);</code>
Clear a bit	Make the selected bit 0	<code>REGISTER &= ~(1 << BIT);</code>	<code>DDRB &= ~(1 << DDB5);</code>
Toggle a bit (software)	Reverse the current bit value	<code>REGISTER ^= (1 << BIT);</code>	<code>PORTB ^= (1 << PORTB5);</code>
Toggle a bit (AVR hardware)	Toggle an output pin in one instruction	<code>PINx = (1 << PINxn);</code>	<code>PINB = (1 << PINB5);</code>
Test a bit	Check whether a bit is set	<code>REGISTER & (1 << BIT)</code>	<code>if (PIND & (1 << PIND2))</code>
Set multiple bits	Set several bits at once	<code>REGISTER</code>	<code>= mask1 mask2 ...</code>

Tip: These five operations—set, clear, toggle, test, and combine multiple bits—are among the most frequently used techniques in embedded programming. Mastering them will enable you to work confidently not only with GPIO registers, but also with timers, communication peripherals, interrupt controllers, analog-to-digital converters, and virtually every other hardware peripheral found in AVR microcontrollers.

PORTx in Input Mode

Many beginners believe PORTx only controls outputs.

It actually has two completely different meanings.

If DDR bit equals

```
1
PORT = Output value
```

If DDR bit equals

```
0
PORT = Pull-up resistor enable
```

This elegant reuse of hardware makes AVR remarkably efficient.

Example

```
DDRD &= ~(1<<DDRD2); // clear bit
PORTD |= (1<<PORTD2); // set bit Enables pull up resistor
```

The pin becomes

Input with Internal Pull-up. Usually writing 1 to port register set the pin high, but if the pin is set for input, a writing 1 enables internal pullup and keeps the pin in input state.

Illustration

A truth table

DDRPOR	T	Result
0	0	Floating Input
0	1	Input with Pull-up
1	0	Output LOW
1	1	Output HIGH

This is probably the most useful illustration in the chapter.

PINx – Reading Inputs

The PIN register always reports the current voltage present on the pin.

```
if (PIND & (1<<PIND2))
```

means

Is PD2 HIGH?

If yes,

execute the following code.

Unlike PORT,

PIN cannot be used to drive outputs. Its primary purpose is reading hardware.